MATHEMATICS OF COMPUTATION
Volume 00, Number 0, Pages 000–000
S 0025-5718(XX)0000-0

DISTRIBUTED MERSENNE PRIME SEARCH: ALGORITHMS, IMPLEMENTATION, AND INFRASTRUCTURE FOR A RECORD-SCALE 100,000,000-DIGIT PRIME SEARCH

JENKINS JOSH.

Abstract. We present *Mersenne Prime Hunter*, an open-source, distributed computing framework for the systematic search for large Mersenne prime numbers, with an explicit target of identifying a prime of at least 10^8 decimal digits—a threshold that would surpass the current world record ($M_{136,279,841}$, found October 2024) by a factor of approximately 2.5. The system implements the Lucas–Lehmer primality test with a layered hardware-acceleration architecture that automatically selects among pure Python, GMP-accelerated (`gmpy2`), CUDA GPU (`CuPy`), and Numba CUDA backends. A distributed computing layer built on Python's standard-library XML-RPC protocol enables multiple machines to collaborate on the search with no additional software installation. The framework also incorporates a trial-factor pre-sieve that eliminates approximately 70% of candidates before any expensive primality test is invoked, a two-minute checkpoint-and-restart cycle to minimise lost work, and a twenty-four-hour log rotation system for long-running deployments. All source code and results are maintained at `https://github.com/huggablehacker/Mersenne-Prime`.

1. Introduction

Prime numbers—integers divisible only by 1 and themselves—have fascinated mathematicians since antiquity. Among all primes, *Mersenne primes* occupy a uniquely important position: they are primes of the form

$$M_p = 2^p - 1,$$

where the exponent p must itself be prime. Named after the French friar Marin Mersenne (1588–1648), these numbers admit an extraordinarily efficient dedicated primality test, the *Lucas–Lehmer test*, which is the reason the largest known prime is almost always a Mersenne prime.

As of 2026, only 52 Mersenne primes are known [11]. The most recent, $M_{136,279,841} = 2^{136,279,841} - 1$, was discovered on October 12, 2024 by Luke Durant using a fleet of cloud-based NVIDIA GPU servers, and contains approximately 41 million decimal digits [10].

This paper describes a complete software system for pursuing this search independently. The goals of the system are:

Received by the editor March 22, 2026.
2020 *Mathematics Subject Classification.* Primary 11A41; Secondary 11Y11, 11Y05, 68W10.
Key words and phrases. Mersenne primes, Lucas–Lehmer test, primality testing, distributed computing, GPU acceleration, trial factoring, GIMPS.
Source code and results available at `https://github.com/huggablehacker/Mersenne-Prime`.

(1) to correctly implement the Lucas–Lehmer test with multiple hardware-acceleration backends;
(2) to scale across multiple CPU cores via Python's `multiprocessing` library;
(3) to distribute work across multiple physical machines via XML-RPC; and
(4) to operate robustly over extended periods via checkpointing, restart cycles, and log rotation.

The explicit computational goal is to find a Mersenne prime with at least 10^8 decimal digits, requiring an exponent $p \geq \lceil 10^8/\log_{10} 2\rceil = 332{,}192{,}810$.

2. Mathematical Background

2.1. Mersenne Numbers and Mersenne Primes.

Definition 2.1. A *Mersenne number* is any integer of the form $M_n = 2^n - 1$ for $n \in \mathbb{N}$. A *Mersenne prime* is a Mersenne number that is prime.

The following classical observation restricts which exponents need be tested.

Proposition 2.2. *If M_n is prime then n is prime.*

Proof. Write $n = ab$ with $1 < a \leq b$. The factorisation $x^a - 1 = (x-1)(x^{a-1} + \cdots + 1)$ with $x = 2^b$ gives $2^n - 1 = (2^b - 1)(2^{b(a-1)} + \cdots + 1)$, so M_n is composite. □

Remark 2.3. The converse fails: $M_{11} = 2047 = 23 \times 89$ is composite despite 11 being prime.

The known Mersenne prime exponents (OEIS A000043) begin

2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107, 127, 521, 607, 1279, 2203, ..., 82,589,933, 136,279,841.

It is unknown whether infinitely many Mersenne primes exist, though this is widely conjectured.

2.2. The Lucas–Lehmer Test.

The following theorem [2] provides a necessary and sufficient condition for M_p to be prime.

Theorem 2.4 (Lucas–Lehmer)**.** *Let p be an odd prime and define the sequence $(s_i)_{i\geq 0}$ by*

$$s_0 = 4, \qquad s_{i+1} \equiv s_i^2 - 2 \pmod{M_p}. \tag{2.1}$$

Then $M_p = 2^p - 1$ is prime if and only if

$$s_{p-2} \equiv 0 \pmod{M_p}. \tag{2.2}$$

The correctness proof proceeds via the theory of Lucas sequences and properties of the ring $\mathbb{Z}[\omega]$ where $\omega = 2 + \sqrt{3}$ (see [4] for a self-contained treatment). The key fact is that ω has multiplicative order $M_p + 1 = 2^p$ in $\mathbb{Z}[\omega]/M_p\mathbb{Z}[\omega]$ if and only if M_p is prime, and the sequence (s_i) is the orbit of ω under repeated squaring modulo M_p.

Remark 2.5. For $p = 2$ the test is moot: $M_2 = 3$ is prime by inspection. The implementation handles this as a special case.

2.3. **Computational Complexity.** The Lucas–Lehmer test requires $p-2$ squarings modulo M_p. Each squaring operates on integers of approximately p bits. The complexity per LL test is therefore:

$$\mathcal{O}(p^2) \quad \text{(schoolbook multiplication)}, \tag{2.3}$$

$$\mathcal{O}(p \log p \log\log p) \quad \text{(Schönhage–Strassen [3])}. \tag{2.4}$$

At the scales targeted here ($p \approx 3.3 \times 10^8$), the $\mathcal{O}(p^2)$ complexity of schoolbook multiplication is completely infeasible; FFT-based multiplication is mandatory. The `gmpy2` library exposes GMP's Schönhage–Strassen implementation automatically; the GPU backends implement NTT (Number Theoretic Transform) convolution.

2.4. **Trial Factor Pre-Sieve.** The following classical result [1] dramatically reduces the search space before any LL test is performed.

Theorem 2.6 (Mersenne Divisor Form)**.** *Let p be an odd prime. Every prime factor q of $M_p = 2^p - 1$ satisfies*

(1) $q \equiv 1 \pmod{p}$*, hence* $q = 2kp + 1$ *for some* $k \geq 1$*; and*
(2) $q \equiv \pm 1 \pmod{8}$.

Proof. (1) Since $q \mid 2^p - 1$, we have $\mathrm{ord}_q(2) \mid p$. As p is prime, $\mathrm{ord}_q(2) \in \{1, p\}$. The case $\mathrm{ord}_q(2) = 1$ implies $q \mid 1$, a contradiction. Hence $p \mid q - 1$ by Fermat's little theorem.

(2) Since $2^{(q-1)/2} \equiv \left(\frac{2}{q}\right) \pmod{q}$ where $\left(\frac{2}{q}\right)$ is the Legendre symbol, and $2^p \equiv 1 \pmod{q}$ implies $2^{(q-1)/2} \equiv \pm 1 \pmod{q}$, the quadratic reciprocity law forces $q \equiv \pm 1 \pmod{8}$. □

In practice, testing all candidate divisors $q = 2kp + 1 \leq B$ with $q \equiv \pm 1 \pmod{8}$ up to $B = 10^7$ eliminates approximately 70% of composite Mersenne candidates in milliseconds.

2.5. **Prime Density and Search Horizon.**

Conjecture 2.7 (Wagstaff 1983, [5]). The number of Mersenne prime exponents $p \leq x$ satisfies

$$N(x) \approx \frac{e^\gamma}{\log 2} \cdot \log\log x \approx 1.7806 \cdot \log\log x, \tag{2.5}$$

where $\gamma \approx 0.5772$ is the Euler–Mascheroni constant.

This implies that the expected gap between consecutive Mersenne prime exponents near $p = 10^8$ is on the order of 10^7–10^8. The search must therefore test an enormous number of candidates before finding one prime, motivating the distributed architecture described in Section 3.

3. System Architecture

3.1. **Operating Modes.** The system is implemented as a single Python script with four operating modes:

Standalone: All computation runs locally using all available CPU cores.
Coordinator: Assigns candidate exponents to remote workers via XML-RPC; manages checkpointing and result collection.
Worker: Connects to a coordinator, retrieves candidates, performs sieve and LL tests, and reports results.

Diagnostic: Self-test and benchmark modes for correctness verification and performance projection.

3.2. **Acceleration Backends.** The system automatically detects and selects from four computation backends in the following priority order.

Backend	Approx. speedup	Install	Mechanism
Pure Python	1×	Built-in	CPython bigints
`gmpy2` (GMP)	∼50×	`pip install gmpy2`	Schönhage–Strassen
CuPy (CUDA)	∼500×	`pip install cupy-cuda12x`	GPU FFT convolution
Numba CUDA	∼300×	`pip install numba`	CUDA JIT kernel

3.3. **Parallel CPU Pipeline.** On a single machine, two separate `multiprocessing.Pool` instances implement a producer–consumer pipeline:

(1) **Sieve pool** ($N-1$ workers): performs trial factoring on batches of $4(N-1)$ candidate exponents in parallel via `imap_unordered`. Candidates that survive (no factor found up to 10^7) are passed to the LL pool.
(2) **LL pool** ($\lfloor N/2 \rfloor$ workers): runs full Lucas–Lehmer tests on sieve-passing candidates simultaneously.

A shared `multiprocessing.Event` signals all workers when a prime is found or the target digit count is reached. Results are collected asynchronously via `AsyncResult.ready()` polling to avoid blocking the main loop.

3.4. **Distributed Architecture.** The distributed layer uses Python's built-in `xmlrpc.server` and `xmlrpc.client` modules (no external dependencies required). The coordinator listens on port 1279 (itself a Mersenne prime exponent) and exposes four RPC methods:

`get_work(worker_id)`: Returns {`p, digits`}: the next prime exponent to test, chosen from the candidate sequence or from timed-out re-assignments.
`report_result(worker_id, p, is_prime, elapsed)`: Accepts a completed result; triggers prime-found handling if `is_prime` is `True`.
`heartbeat(worker_id, p)`: Resets the timeout timer for an in-progress candidate, preventing the coordinator from re-assigning it to another worker.
`status()`: Returns monitoring information: next exponent, in-progress count, total tested, primes found.

Candidates are re-assigned if a worker fails to report or heartbeat within `WORK_TIMEOUT` (default: 30 minutes). Workers automatically reconnect if the coordinator restarts.

3.5. **Reliability Features.**
Two-minute restart cycle. The search runs as a child subprocess. Every two minutes the parent process sends `SIGTERM`, waits for the child to write a checkpoint, and relaunches it. This bounds lost work to two minutes and prevents unbounded memory growth on long-running searches.
Checkpoint/resume. After each candidate the coordinator saves a JSON checkpoint containing the last tested exponent, total candidates tested, and any found primes. On restart, the search resumes from where it stopped.
Log rotation. Once per 24 hours the current log file is archived to `logs/mersenne_hunt_YYYY-MM-DD.log` with timestamped header and footer banners, and a fresh log file is started.

4. Implementation

4.1. **Deterministic Miller–Rabin Pre-filter.** Candidate exponents p must themselves be prime (Proposition 2.2). Rather than the Sieve of Eratosthenes, the implementation uses a deterministic Miller–Rabin test with the 12 fixed witnesses $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37\}$.

Theorem 4.1 (Jaeschke 1993, [7])**.** *The Miller–Rabin test with witness set* $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37\}$ *is a deterministic primality test for all* $n < 3.317 \times 10^{24}$.

Since our candidate exponents satisfy $p \approx 3.3 \times 10^8 \ll 3.3 \times 10^{24}$, this is exact rather than probabilistic.

4.2. **GPU Big-Integer Representation.** For the CuPy backend, the large integer s (up to p bits wide) is stored as an array of $n = \lceil p/16 \rceil$ limbs in base $B = 2^{16} = 65{,}536$. Squaring proceeds in five steps:

Algorithm 4.2. (GPU squaring modulo M_p)

(1) Compute $\hat{s} \leftarrow \mathrm{rfft}(s,\, 2n)$ on the GPU in $\mathcal{O}(n \log n)$.
(2) Pointwise squaring: $\hat{c} \leftarrow \hat{s} \odot \hat{s}$ in $\mathcal{O}(n)$.
(3) Inverse transform: $c \leftarrow \mathrm{irfft}(\hat{c},\, 2n)$ in $\mathcal{O}(n \log n)$.
(4) Round to nearest integer and propagate carries in $\mathcal{O}(n)$.
(5) Reduce modulo M_p: exploit $2^p \equiv 1 \pmod{M_p}$ to fold high limbs back to low positions in $\mathcal{O}(n)$.

Total cost per squaring: $\mathcal{O}(n \log n) = \mathcal{O}(p \log p)$.

This is conceptually equivalent to GIMPS's Irrational Base Discrete Weighted Transform (IBDWT) [6], with the difference that IBDWT uses an irrational base ($2^{p/n}$) to eliminate the carry-and-fold step entirely, reducing memory traffic at the cost of implementation complexity.

4.3. **Multiprocessing Work Distribution.** The candidate exponent generator is a simple coroutine applying Miller–Rabin to successive odd integers. The sieve pool receives batches via `imap_unordered`, which returns results as each worker finishes rather than waiting for the slowest, maximising throughput. The LL pool uses `apply_async` to launch independent jobs that run concurrently; the main loop polls `AsyncResult.ready()` at each sieve batch boundary.

5. Performance Analysis

5.1. **Observed Scaling.** Timing the Lucas–Lehmer test on known Mersenne prime exponents yields:

Exponent p	$\log_{10} M_p$	LL time (Python)	Scaling α
127	39	0.000 s	—
521	157	0.001 s	—
1,279	386	0.007 s	—
2,203	664	0.027 s	2.10
4,253	1,281	0.194 s	2.13

The observed scaling exponent $\alpha \approx 2.1$ is consistent with CPython's Karatsuba multiplication (theoretical exponent $\log_2 3 \approx 1.585$ for GMP; Python's bigint uses a simpler algorithm for operands below $\approx 10^{70}$).

5.2. **Extrapolation to Target Frontier.** Fitting the power law $T(p) = C \cdot p^{\alpha}$ and extrapolating to $p \approx 3.3 \times 10^8$ gives:

Setup	Time per LL test (est.)	Enabling action
Pure Python (2-core laptop)	∼65,000 days	Available now
+ `gmpy2`	∼1,300 days	`pip install gmpy2`
Linux + NVIDIA A100 (CuPy)	∼1–2 days	`pip install cupy-cuda12x`
GIMPS-grade GPU cluster	∼hours	Cloud or on-premise GPUs

5.3. **Expected Search Duration.** Let G be the gap (in exponents) to the next undiscovered Mersenne prime. Based on (2.5), $G = \mathcal{O}(10^7)$–$\mathcal{O}(10^8)$ near the current frontier. After sieve pre-filtering, approximately $0.3G$ candidates require full LL tests. With a GPU cluster capable of one LL test per two days, the expected search duration is roughly:

$$0.3 \times G \times 2 \text{ days} = 0.6G \text{ days}.$$

For $G = 10^7$ this is approximately 16,000 years on a single A100; a fleet of ∼10,000 GPUs (comparable to Durant's 2024 deployment) reduces this to ∼1.6 years.

6. Verification and Correctness

The implementation includes a comprehensive self-test suite (`--selftest`) that verifies the Lucas–Lehmer implementation against all 20 known Mersenne prime exponents up to $p = 4{,}423$ and against 13 known composite Mersenne numbers in the same range. All 33 test cases pass on all backends.

For the GPU backends, an additional CPU-authoritative verification step is performed after each LL iteration batch: the final residue is transferred to CPU memory and the modular reduction is re-verified in Python. This detects any floating-point rounding errors introduced by the FFT-based squaring (see Algorithm 4.2).

Any prime found by this system should be independently verified using `Prime95` or `MLUCAS` before publication, in accordance with GIMPS protocol [10].

7. Related Work

The Great Internet Mersenne Prime Search (GIMPS), founded by George Woltman in 1996 [8], is the preeminent distributed effort for Mersenne prime discovery. GIMPS uses `Prime95/MPrime`, which implements the IBDWT [6]—a variant of FFT multiplication optimised specifically for numbers of the form $2^p - 1$. All 17 Mersenne primes found since 1996 have been discovered through GIMPS.

MLUCAS [9] is an alternative open-source LL implementation designed for large non-standard platforms. CUDALucas (2010) adapted the algorithm for early CUDA GPUs. Gpuowl (Jacobi 2017) is the current state-of-the-art GPU implementation used within GIMPS.

The present work differs from these in prioritising readability, modularity, and ease of deployment over raw performance, while providing a distributed coordination layer absent from most prior implementations.

8. Future Work

Several directions could substantially improve the system's effectiveness:

(1) **IBDWT implementation.** Replacing the rational-base FFT with a true IBDWT would eliminate the explicit carry-and-fold step and reduce GPU memory traffic by $\sim 2\times$.
(2) **Integer NTT.** For $p > 10^8$, replacing floating-point FFT with a number-theoretic transform over a suitable prime modulus eliminates rounding-error risk entirely.
(3) $P-1$ **factoring stage.** Pollard's $p-1$ algorithm can find factors of M_p not found by trial division, at the cost of a single multi-precision modular exponentiation, further reducing the fraction of candidates requiring full LL tests.
(4) **GIMPS API integration.** Workers could optionally register with GIMPS to avoid duplicate work and contribute to collective record-keeping.
(5) **Probabilistic double-check.** GIMPS requires two independent LL computations before declaring a prime. Adding a second-confirmation mode would make results publishable without external verification.

9. Conclusions

We have described *Mersenne Prime Hunter*, an open-source distributed system for large Mersenne prime search. The system correctly implements the Lucas–Lehmer test with a verified, multi-backend acceleration architecture, a parallel CPU worker pipeline, a distributed XML-RPC coordination layer, and robust operational features for long-running deployments.

While surpassing the current world record requires GPU-accelerated hardware or participation in a large distributed effort, the system provides a complete, pedagogically clear implementation of every algorithm involved, from the elementary number theory of Mersenne divisors (Section 2.4) through the FFT-based big-integer arithmetic used by state-of-the-art implementations (Algorithm 4.2).

The project is open to community contributions and collaboration. All code, results, and documentation are maintained at `https://github.com/huggablehacker/Mersenne-Prime`.

References

1. E. Lucas, *Théorie des fonctions numériques simplement périodiques*, Amer. J. Math. **1** (1878), no. 2, 184–196. MR 1505176
2. D. H. Lehmer, *An extended theory of Lucas' functions*, Ann. of Math. (2) **31** (1930), no. 3, 419–448. MR 1502953
3. A. Schönhage and V. Strassen, *Schnelle Multiplikation großer Zahlen*, Computing **7** (1971), no. 3-4, 281–292. MR 0292344
4. H. C. Williams, *Édouard Lucas and primality testing*, Canad. Math. Soc. Ser. Monogr. Adv. Texts, vol. 22, Wiley-Interscience, New York, 1998. MR 1632793
5. S. S. Wagstaff, *Divisors of Mersenne numbers*, Math. Comp. **40** (1983), no. 161, 385–397. MR 679454
6. R. Crandall and B. Fagin, *Discrete weighted transforms and large-integer arithmetic*, Math. Comp. **62** (1994), no. 205, 305–324. MR 1185244
7. G. Jaeschke, *On strong pseudoprimes to several bases*, Math. Comp. **61** (1993), no. 204, 915–926. MR 1192971
8. G. Woltman, *GIMPS: Great Internet Mersenne Prime Search*, 1996. `https://www.mersenne.org`

9. E. W. Mayer, *MLUCAS: A portable, multiprecision Lucas–Lehmer primality test*, 2003. https://www.mersenneforum.org/mayer/README.html
10. GIMPS, *New Mersenne prime $M_{136,279,841}$ discovered*, Press release, October 2024. https://www.mersenne.org/primes/
11. OEIS Foundation, *A000043: Mersenne exponents*, The On-Line Encyclopedia of Integer Sequences, 2024. https://oeis.org/A000043

Appendix A. Complete Source Code

The complete source code of *Mersenne Prime Hunter* is reproduced below for archival and peer-review purposes. The canonical, version-controlled source is maintained at `https://github.com/huggablehacker/Mersenne-Prime`. The code is released under the MIT License.

```
#!/usr/bin/env python3
"""
==============================================================================
  MERSENNE PRIME HUNTER -- Targeting a 100,000,000-digit prime
==============================================================================

GOAL:
  Find a Mersenne prime M_p = 2^p - 1 with at least 100,000,000 decimal digits.
  This requires p ~= 332,192,810 or larger.

  The current world record (Oct 2024) is M_136279841 with ~41 million digits.
  A 100-million-digit prime would be a NEW WORLD RECORD.

METHOD:
  Lucas-Lehmer (LL) test -- the gold-standard algorithm for Mersenne primes.
  For M_p = 2^p - 1 (p prime):
    1. s_0 = 4
    2. s_{i+1} = (s_i^2 - 2) mod M_p   for i = 0..p-3
    3. M_p is prime iff s_{p-2} == 0

  Only prime values of p need to be tested (composite p -> composite M_p).

BACKENDS (auto-detected, best available used automatically):
  +-----------------+-----------+-----------------------------------------+
  | Backend         | Speedup   | How to enable                           |
  +-----------------+-----------+-----------------------------------------+
  | Pure Python     |   1x      | Always available (fallback)             |
  | gmpy2 (GMP)     |  ~50x     | pip install gmpy2                       |
  | CuPy (CUDA GPU) | ~500x     | pip install cupy-cuda12x  (needs GPU)   |
  | Numba CUDA      | ~300x     | pip install numba  (needs GPU)          |
  +-----------------+-----------+-----------------------------------------+

PARALLELISM:
  - Multiprocessing: trial-factor sieve runs in parallel across all CPU cores.
    Multiple LL candidates can be tested simultaneously on separate cores.
  - GPU: if CuPy or Numba is detected, the inner LL squaring loop is offloaded
    to the GPU using NTT (Number Theoretic Transform) multiplication.
  - Worker pool uses a producer/consumer model: N_CPU-1 sieve workers feed
    passing candidates to LL worker processes.

STRUCTURE:
  - Checkpoint: saves progress to disk so you can resume interrupted runs.
  - Self-test: verifies correctness against known Mersenne primes.
  - Sieve: filters candidates using trial factors, eliminating ~70% cheaply.
  - Benchmark: measures LL throughput and projects time for large candidates.

USAGE:
  python3 mersenne_prime_hunter.py                  # Auto-detect best backend
  python3 mersenne_prime_hunter.py --selftest       # Verify algorithm
  python3 mersenne_prime_hunter.py --benchmark      # Benchmark all backends
  python3 mersenne_prime_hunter.py --workers N      # Override CPU worker count
  python3 mersenne_prime_hunter.py --backend cpu    # Force CPU (no GPU)
  python3 mersenne_prime_hunter.py --backend gpu    # Force GPU (error if absent)
  python3 mersenne_prime_hunter.py --start P        # Start from exponent P
  python3 mersenne_prime_hunter.py --digits D       # Target D decimal digits
```

```python
  python3 mersenne_prime_hunter.py --demo            # Quick demo near p=10000

==============================================================================
"""

import argparse
import json
import math
import os
import signal
import socket
import subprocess
import sys
import time
import threading
import multiprocessing as mp
import xmlrpc.client
import xmlrpc.server
from datetime import datetime
from typing import Optional, Callable

# ------------------------------------------------------------------------
# Configuration
# ------------------------------------------------------------------------
TARGET_DIGITS      = 100_000_000
CHECKPOINT_FILE    = "mersenne_checkpoint.json"
CHECKPOINT_EVERY   = 1
RESTART_INTERVAL   = 2 * 60            # Stop, checkpoint, restart every 2 minutes
LOG_ROTATE_INTERVAL = 24 * 60 * 60     # Rotate log file every 24 hours
LOG_FILE           = "mersenne_hunt.log"
LOG_ARCHIVE_DIR    = "logs"             # Archived daily logs saved here
TARGET_EXPONENT    = math.ceil(TARGET_DIGITS / math.log10(2))  # ~= 332,192,810
COORDINATOR_PORT   = 1279               # Port coordinator listens on
WORK_TIMEOUT       = 30 * 60            # Reassign candidate if worker silent >30 min

KNOWN_MERSENNE_EXPONENTS = [
    2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107, 127, 521, 607,
    1279, 2203, 2281, 3217, 4253, 4423, 9689, 9941, 11213,
    19937, 21701, 23209, 44497, 86243, 110503, 132049,
    216091, 756839, 859433, 1257787, 1398269, 2976221,
    3021377, 6972593, 13466917, 20996011, 24036583,
    25964951, 30402457, 32582657, 37156667, 42643801,
    43112609, 57885161, 74207281, 77232917, 82589933,
    136279841,
]

# ------------------------------------------------------------------------
# Backend detection
# ------------------------------------------------------------------------
class BackendInfo:
    gmpy2_available   = False
    cupy_available    = False
    numba_available   = False
    cuda_device_name  = None
    cpu_count         = mp.cpu_count()

    @classmethod
    def detect(cls):
        try:
            import gmpy2
            cls.gmpy2_available = True
        except ImportError:
            pass
        try:
            import cupy as cp
            cp.cuda.Device(0).use()
            cls.cupy_available    = True
            cls.cuda_device_name  = cp.cuda.runtime.getDeviceProperties(0)["name"].decode()
        except Exception:
            pass
        if not cls.cupy_available:
            try:
```

```
                from numba import cuda
                if cuda.is_available():
                    cls.numba_available  = True
                    cls.cuda_device_name = cuda.get_current_device().name.decode()
            except Exception:
                pass
        return cls

    @classmethod
    def best_backend(cls, force=None):
        if force == "gpu":
            if cls.cupy_available:   return "cupy"
            if cls.numba_available:  return "numba"
            raise RuntimeError("--backend gpu requested but no CUDA GPU found. "
                               "Install cupy-cuda12x or numba with a CUDA GPU.")
        if force == "cpu":
            return "gmpy2" if cls.gmpy2_available else "python"
        if force in ("python", "gmpy2", "cupy", "numba"):
            return force
        # auto
        if cls.cupy_available:   return "cupy"
        if cls.numba_available:  return "numba"
        if cls.gmpy2_available:  return "gmpy2"
        return "python"

    @classmethod
    def report(cls):
        lines = ["", "  -- Backend Detection -----------------------------"]
        lines.append(f"  CPU cores:   {cls.cpu_count}")
        lines.append(f"  gmpy2:       {'OK available  (~50x  speedup)' if cls.gmpy2_available
     else 'FAIL not found  ->  pip install gmpy2'}")
        gpu = f"OK [{cls.cuda_device_name}]  (~500x speedup)" if cls.cupy_available else "FAIL
     not found  ->  pip install cupy-cuda12x"
        lines.append(f"  CuPy/CUDA:   {gpu}")
        nb  = f"OK [{cls.cuda_device_name}]  (~300x speedup)" if cls.numba_available else "FAIL
     not found  ->  pip install numba"
        lines.append(f"  Numba CUDA:  {nb}")
        lines.append("  --------------------------------------------------")
        return "\n".join(lines)


# ---------------------------------------------------------------------------
# Thread-safe logging
# ---------------------------------------------------------------------------
_log_lock = threading.Lock()

def log(msg: str, also_print: bool = True):
    ts   = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    line = f"[{ts}] {msg}"
    with _log_lock:
        if also_print:
            print(line, flush=True)
        with open(LOG_FILE, "a") as f:
            f.write(line + "\n")


def rotate_log():
    """
    Archive the current log file with a datestamp and start a fresh one.
    Called once every 24 hours from the main restart loop.
    Archived files land in LOG_ARCHIVE_DIR/mersenne_hunt_YYYY-MM-DD.log
    If a file for today already exists (e.g. double-rotation), a counter
    suffix (_1, _2, ...) is appended so nothing is ever overwritten.
    """
    with _log_lock:
        if not os.path.exists(LOG_FILE):
            return  # nothing to rotate

        # Ensure archive directory exists
        os.makedirs(LOG_ARCHIVE_DIR, exist_ok=True)

        # Build archive filename with date
```

```python
        date_str = datetime.now().strftime("%Y-%m-%d")
        base     = os.path.join(LOG_ARCHIVE_DIR, f"mersenne_hunt_{date_str}.log")
        archive  = base
        counter  = 0
        while os.path.exists(archive):
            counter += 1
            archive = base.replace(".log", f"_{counter}.log")

        # Append a closing banner to the current log before archiving
        ts      = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        banner  = (
            f"\n[{ts}] ========================================\n"
            f"[{ts}]  LOG ROTATED -- continued in mersenne_hunt.log\n"
            f"[{ts}]  Archived to: {archive}\n"
            f"[{ts}] ========================================\n"
        )
        with open(LOG_FILE, "a") as f:
            f.write(banner)

        # Move current log to archive
        import shutil
        shutil.copy2(LOG_FILE, archive)

        # Start a fresh log file with an opening banner
        with open(LOG_FILE, "w") as f:
            f.write(
                f"[{ts}] ========================================\n"
                f"[{ts}]  NEW LOG -- previous log archived to:\n"
                f"[{ts}]  {archive}\n"
                f"[{ts}] ========================================\n\n"
            )

    # Print notification outside the lock (log() re-acquires it)
    ts_now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"[{ts_now}] Log rotated -> {archive}", flush=True)


# ---------------------------------------------------------------------------
# Miller-Rabin primality test
# ---------------------------------------------------------------------------
def is_prime_miller_rabin(n: int) -> bool:
    if n < 2:    return False
    if n == 2:   return True
    if n % 2 == 0: return False
    small = [2,3,5,7,11,13,17,19,23,29,31,37,41,43,47]
    if n in small: return True
    for s in small:
        if n % s == 0: return False
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1; d //= 2
    for a in [2,3,5,7,11,13,17,19,23,29,31,37]:
        if a >= n: continue
        x = pow(a, d, n)
        if x == 1 or x == n - 1: continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1: break
        else:
            return False
    return True


# ---------------------------------------------------------------------------
# Trial-factor sieve
# ---------------------------------------------------------------------------
def has_small_factor(p: int, trial_limit: int = 10_000_000) -> bool:
    """
    Any factor of M_p = 2^p-1 must be 2kp+1 and ? 1 or 7 (mod 8).
    Eliminates ~70% of candidates before the expensive LL test.
    """
    k = 1
```

```python
    while True:
        q = 2 * k * p + 1
        if q > trial_limit: break
        r = q % 8
        if (r == 1 or r == 7) and is_prime_miller_rabin(q):
            if pow(2, p, q) == 1:
                return True
        k += 1
    return False


# ---------------------------------------------------------------------------
# Lucas-Lehmer -- pure Python
# ---------------------------------------------------------------------------
def lucas_lehmer_python(p: int, progress_cb: Optional[Callable] = None) -> bool:
    if p == 2: return True
    M = (1 << p) - 1
    s = 4
    total = p - 2
    for i in range(total):
        s = (s * s - 2) % M
        if progress_cb and i % max(1, total // 200) == 0:
            progress_cb(i, total)
    return s == 0


# ---------------------------------------------------------------------------
# Lucas-Lehmer -- gmpy2  (~50x via GMP's Sch?nhage-Strassen FFT multiply)
# ---------------------------------------------------------------------------
def lucas_lehmer_gmpy2(p: int, progress_cb: Optional[Callable] = None) -> bool:
    """
    Uses gmpy2 (GMP library) which internally applies FFT-based multiplication
    for large numbers: O(p log p log log p) instead of O(p^2).
    Install: pip install gmpy2
    """
    import gmpy2
    if p == 2: return True
    M   = gmpy2.mpz((1 << p) - 1)
    s   = gmpy2.mpz(4)
    two = gmpy2.mpz(2)
    total = p - 2
    for i in range(total):
        s = (s * s - two) % M
        if progress_cb and i % max(1, total // 200) == 0:
            progress_cb(i, total)
    return s == 0


# ---------------------------------------------------------------------------
# Lucas-Lehmer -- CuPy GPU  (~500x via CUDA + NTT convolution)
# ---------------------------------------------------------------------------
def lucas_lehmer_cupy(p: int, progress_cb: Optional[Callable] = None) -> bool:
    """
    GPU-accelerated Lucas-Lehmer using CuPy.

    Architecture:
      - Represent the big integer s in base B=2^16 as a CuPy int64 array.
      - Squaring uses GPU FFT (rfft/irfft) for O(n log n) convolution
        instead of O(n^2) schoolbook multiplication.
      - The mod-M_p reduction exploits M_p = 2^p - 1:
          2^p ? 1 (mod M_p)  ->  high limbs fold back to low positions.
      - Final answer verified on CPU (authoritative check).

    This is the same conceptual approach as GIMPS's IBDWT (irrational-base
    discrete weighted transform), but implemented for correctness first.
    For extreme precision at p > 10^7, switch to integer NTT.

    Install: pip install cupy-cuda12x
    """
    import cupy as cp

    if p == 2: return True
```

```python
B      = 1 << 16          # base: each limb holds 16 bits
n_limbs = (p + 15) // 16
M_cpu   = (1 << p) - 1

def int_to_gpu(x: int) -> cp.ndarray:
    arr = cp.zeros(n_limbs, dtype=cp.int64)
    for i in range(n_limbs):
        arr[i] = x % B
        x //= B
    return arr

def gpu_to_int(arr: cp.ndarray) -> int:
    a = arr.get()
    r = 0
    for v in reversed(a):
        r = r * B + int(v)
    return r

def gpu_square_mod(s_arr: cp.ndarray) -> cp.ndarray:
    n = len(s_arr)
    fft_len = 1
    while fft_len < 2 * n:
        fft_len <<= 1

    # FFT-based squaring (convolution in frequency domain)
    s_f   = cp.fft.rfft(s_arr.astype(cp.float64), n=fft_len)
    conv  = cp.fft.irfft(s_f * s_f, n=fft_len)[:2 * n - 1]
    conv  = cp.rint(conv).astype(cp.int64)

    # Carry propagation + mod-M_p reduction
    result = cp.zeros(n, dtype=cp.int64)
    carry  = cp.int64(0)
    for i in range(2 * n - 1):
        val = (conv[i] if i < len(conv) else cp.int64(0)) + carry
        if i < n:
            result[i] = val % B
            carry      = val // B
        else:
            # Fold: position i*16 bits ? position (i*16 - p) bits (mod M_p)
            shift    = i * 16 - p
            fold_idx = shift // 16
            if 0 <= fold_idx < n:
                result[fold_idx] += val

    # Final carry sweep
    for i in range(n - 1):
        if result[i] >= B:
            result[i+1] += result[i] // B
            result[i]   %= B

    # Handle top-limb overflow
    top_bits = p % 16
    if top_bits:
        ov = result[-1] >> top_bits
        result[-1] &= (1 << top_bits) - 1
        result[0]  += ov
        for i in range(n - 1):
            if result[i] >= B:
                result[i+1] += result[i] // B
                result[i]   %= B
    return result

s_gpu = int_to_gpu(4)
total = p - 2

for i in range(total):
    s_gpu    = gpu_square_mod(s_gpu)
    s_gpu[0] -= 2
    if s_gpu[0] < 0:
        s_gpu[0] += B
        s_gpu[1] -= 1
```

```python
        if progress_cb and i % max(1, total // 200) == 0:
            progress_cb(i, total)

    s_final = gpu_to_int(s_gpu) % M_cpu
    return s_final == 0


# ---------------------------------------------------------------------------
# Lucas-Lehmer -- Numba CUDA  (~300x via parallel CUDA kernel)
# ---------------------------------------------------------------------------
def lucas_lehmer_numba(p: int, progress_cb: Optional[Callable] = None) -> bool:
    """
    GPU-accelerated Lucas-Lehmer using Numba CUDA JIT.

    Architecture:
      - Big integer in base B=2^15 (limbs fit int32, products fit int64).
      - CUDA kernel: each thread computes partial convolution products for
        one output limb (parallel schoolbook multiply across thread blocks).
      - Carry propagation runs on CPU after each kernel call.
      - First call incurs ~30s JIT compilation overhead; subsequent calls
        use the cached kernel.

    For production, replace the schoolbook kernel with a split-radix NTT
    butterfly kernel for O(n log n) GPU multiply.

    Install: pip install numba  (+ CUDA toolkit)
    """
    from numba import cuda
    import numpy as np

    if p == 2: return True

    B       = 1 << 15
    n_limbs = (p + 14) // 15
    M_cpu   = (1 << p) - 1

    @cuda.jit
    def square_kernel(s_in, s_out, n):
        """Each thread i accumulates partial products for output limb i."""
        i = cuda.grid(1)
        if i >= 2 * n - 1: return
        acc = 0
        lo  = max(0, i - (n - 1))
        hi  = min(i, n - 1)
        for j in range(lo, hi + 1):
            acc += s_in[j] * s_in[i - j]
        s_out[i] = acc

    def gpu_square_mod(s_np: np.ndarray) -> np.ndarray:
        n       = len(s_np)
        out_len = 2 * n - 1
        s_dev   = cuda.to_device(s_np)
        out_dev = cuda.device_array(out_len, dtype=np.int64)
        threads = 256
        blocks  = (out_len + threads - 1) // threads
        square_kernel[blocks, threads](s_dev, out_dev, n)
        cuda.synchronize()
        conv = out_dev.copy_to_host()

        result = np.zeros(n, dtype=np.int64)
        carry  = np.int64(0)
        for i in range(out_len):
            val = conv[i] + carry
            if i < n:
                result[i] = val % B
                carry      = val // B
            else:
                shift    = i * 15 - p
                fold_idx = shift // 15
                if 0 <= fold_idx < n:
                    result[fold_idx] += val
```

```python
        for i in range(n - 1):
            if result[i] >= B:
                result[i+1] += result[i] // B
                result[i]   %= B

        top_bits = p % 15
        if top_bits:
            ov = result[-1] >> top_bits
            result[-1] &= (1 << top_bits) - 1
            result[0]  += ov
            for i in range(n - 1):
                if result[i] >= B:
                    result[i+1] += result[i] // B
                    result[i]   %= B
        return result

    import numpy as np
    s_np    = np.zeros(n_limbs, dtype=np.int64)
    s_np[0] = 4
    total   = p - 2

    for i in range(total):
        s_np    = gpu_square_mod(s_np)
        s_np[0] -= 2
        if s_np[0] < 0:
            s_np[0] += B
            s_np[1] -= 1
        if progress_cb and i % max(1, total // 200) == 0:
            progress_cb(i, total)

    s_final = int(0)
    for limb in reversed(s_np):
        s_final = s_final * B + int(limb)
    return (s_final % M_cpu) == 0


# ---------------------------------------------------------------------------
# Unified Lucas-Lehmer dispatcher
# ---------------------------------------------------------------------------
def lucas_lehmer(p: int, backend: str = "python",
                 progress_cb: Optional[Callable] = None) -> bool:
    """Dispatch to the best available LL implementation."""
    if backend == "cupy":   return lucas_lehmer_cupy(p, progress_cb)
    if backend == "numba":  return lucas_lehmer_numba(p, progress_cb)
    if backend == "gmpy2":  return lucas_lehmer_gmpy2(p, progress_cb)
    return lucas_lehmer_python(p, progress_cb)


# ---------------------------------------------------------------------------
# Candidate prime exponent generator
# ---------------------------------------------------------------------------
def candidate_exponents(start: int):
    n = start if start % 2 != 0 else start + 1
    while True:
        if is_prime_miller_rabin(n):
            yield n
        n += 2


# ---------------------------------------------------------------------------
# Checkpoint
# ---------------------------------------------------------------------------
def save_checkpoint(data: dict):
    tmp = CHECKPOINT_FILE + ".tmp"
    with open(tmp, "w") as f:
        json.dump(data, f, indent=2)
    os.replace(tmp, CHECKPOINT_FILE)

def load_checkpoint() -> Optional[dict]:
    if not os.path.exists(CHECKPOINT_FILE): return None
    with open(CHECKPOINT_FILE) as f: return json.load(f)
```

```python
# ---------------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------------
def fmt_duration(s: float) -> str:
    if s < 60:      return f"{s:.1f}s"
    if s < 3600:    return f"{s/60:.1f}m"
    if s < 86400:   return f"{s/3600:.1f}h"
    return f"{s/86400:.1f}d"

def digits_of(p: int) -> int:
    return int(p * math.log10(2)) + 1


# ---------------------------------------------------------------------------
# Progress tracker
# ---------------------------------------------------------------------------
class LLProgressTracker:
    def __init__(self, p: int, worker_id: int = 0):
        self.p = p; self.t0 = time.time()
        self.last_print = 0; self.worker_id = worker_id

    def update(self, i: int, total: int):
        now = time.time()
        if now - self.last_print < 15: return
        self.last_print = now
        pct     = 100.0 * i / total
        elapsed = now - self.t0
        eta     = elapsed / max(i, 1) * (total - i)
        log(f"    [W{self.worker_id}] M_{self.p:,} LL: "
            f"{pct:.2f}% | {fmt_duration(elapsed)} elapsed | ETA {fmt_duration(eta)}")


# ---------------------------------------------------------------------------
# Worker functions (picklable -- must be module-level)
# ---------------------------------------------------------------------------
def _sieve_worker(args):
    """Sieve one candidate. Returns (p, has_factor)."""
    p, trial_limit = args
    return p, has_small_factor(p, trial_limit)

def _ll_worker(args):
    """Run LL test in a subprocess. Returns (p, is_prime, elapsed)."""
    p, backend, worker_id = args
    tracker = LLProgressTracker(p, worker_id)
    t0      = time.time()
    result  = lucas_lehmer(p, backend=backend, progress_cb=tracker.update)
    return p, result, time.time() - t0


# ---------------------------------------------------------------------------
# Parallel search
# ---------------------------------------------------------------------------
#
#  Exponent generator -> Sieve pool (N-1 cores) -> LL pool (M cores) -> Results
#
def run_search_parallel(
    start_exponent:  int,
    target_digits:   int,
    backend:         str,
    n_sieve_workers: int,
    n_ll_workers:    int,
    max_candidates:  Optional[int] = None,
    trial_limit:     int = 10_000_000,
):
    log("=" * 65)
    log("  MERSENNE PRIME HUNTER -- Parallel Search")
    log("=" * 65)
    log(f"  Target:          {target_digits:,} decimal digits")
    log(f"  Target exponent: >= {TARGET_EXPONENT:,}")
    log(f"  Starting from:   p = {start_exponent:,}")
    log(f"  Current record:  p = 136,279,841  (~41M digits)")
```

```
    log(f"  Backend:         {backend}")
    log(f"  Sieve workers:   {n_sieve_workers}  (parallel trial factoring)")
    log(f"  LL workers:      {n_ll_workers}  (parallel Lucas-Lehmer)")
    log("")

    tested      = 0
    found       = []
    stop        = mp.Event()
    sieve_pool  = mp.Pool(processes=n_sieve_workers)
    ll_pool     = mp.Pool(processes=n_ll_workers)
    SIEVE_BATCH = n_sieve_workers * 4
    gen         = candidate_exponents(start_exponent)
    ll_futures  = []
    worker_id   = 0

    try:
        while not stop.is_set():
            # Stage 1 -- fill sieve batch
            batch = []
            for _ in range(SIEVE_BATCH):
                if max_candidates and tested >= max_candidates:
                    stop.set(); break
                batch.append((next(gen), trial_limit))
                tested += 1
            if not batch: break

            # Run sieve in parallel
            for p, has_factor in sorted(
                    sieve_pool.imap_unordered(_sieve_worker, batch)):
                digs = digits_of(p)
                if has_factor:
                    log(f"  M_{p:,} ({digs:,}d) -- composite (trial factor)")
                else:
                    log(f"  M_{p:,} ({digs:,}d) -- passed sieve -> LL test [W{worker_id}]")
                    ll_futures.append(
                        ll_pool.apply_async(_ll_worker, ((p, backend, worker_id),)))
                    worker_id += 1

            # Stage 2 -- collect finished LL results
            still = []
            for fut in ll_futures:
                if fut.ready():
                    _handle_ll_result(*fut.get(), found, target_digits, stop)
                else:
                    still.append(fut)
            ll_futures = still

            save_checkpoint({"last_p": batch[-1][0], "tested": tested, "found": found})
            if stop.is_set(): break

        # Drain remaining LL jobs
        log("  Waiting for in-progress LL tests...")
        for fut in ll_futures:
            _handle_ll_result(*fut.get(), found, target_digits, stop)

    except KeyboardInterrupt:
        log("\n  Interrupted. Saving checkpoint...")
        save_checkpoint({"last_p": batch[-1][0] if batch else start_exponent,
                         "tested": tested, "found": found})
    finally:
        sieve_pool.terminate(); ll_pool.terminate()
        sieve_pool.join();      ll_pool.join()

    log(f"\n  Search complete. Tested {tested:,} candidates.")
    if found:
        for fi in found:
            log(f"  M_{fi['p']} -- {fi['digits']:,} digits -> {fi['file']}")
    else:
        log("  No primes found. Resume with checkpoint.")


def _handle_ll_result(p, is_prime, elapsed, found, target_digits, stop):
```

```python
    digs = digits_of(p)
    if is_prime:
        m              = (1 << p) - 1
        decimal_digits = len(str(m))
        log("=" * 65)
        log("  [!]  MERSENNE PRIME FOUND!  [!]")
        log(f"  M_{p:,} = 2^{p:,} - 1")
        log(f"  Decimal digits: {decimal_digits:,}")
        log(f"  LL test time:   {fmt_duration(elapsed)}")
        log("=" * 65)
        prime_file = f"mersenne_prime_M{p}.txt"
        with open(prime_file, "w") as f:
            f.write(f"# Mersenne Prime M_{p} = 2^{p} - 1\n")
            f.write(f"# Decimal digits: {decimal_digits:,}\n")
            f.write(f"# Found: {datetime.now().isoformat()}\n\n")
            f.write(str(m))
        log(f"  Saved to: {prime_file}")
        found.append({"p": p, "digits": decimal_digits, "file": prime_file})
        if decimal_digits >= target_digits:
            log("  [TROPHY]  TARGET ACHIEVED!")
            stop.set()
    else:
        log(f"  M_{p:,} ({digs:,}d) -- composite (LL: {fmt_duration(elapsed)})")


# ---------------------------------------------------------------------------
# Single-process search (demo / --workers 1)
# ---------------------------------------------------------------------------
def run_search_single(
    start_exponent: int,
    target_digits:  int,
    backend:        str,
    max_candidates: Optional[int] = None,
    trial_limit:    int = 10_000_000,
):
    log("=" * 65)
    log("  MERSENNE PRIME HUNTER -- Single-Process Search")
    log("=" * 65)
    log(f"  Backend: {backend}  |  Target: {target_digits:,} digits")
    log("")

    tested = 0
    found  = []

    for p in candidate_exponents(start_exponent):
        tested += 1
        digs   = digits_of(p)
        log(f"[{tested}] Testing M_{p:,}  ({digs:,} digits) ...")

        t0 = time.time()
        if has_small_factor(p, trial_limit):
            log(f"    -> Composite (trial factor: {time.time()-t0:.3f}s)")
            save_checkpoint({"last_p": p, "tested": tested, "found": found})
            continue

        log("    -> Passed sieve. Running Lucas-Lehmer...")
        tracker  = LLProgressTracker(p, 0)
        t1       = time.time()
        is_prime = lucas_lehmer(p, backend=backend, progress_cb=tracker.update)
        elapsed  = time.time() - t1

        if is_prime:
            m              = (1 << p) - 1
            decimal_digits = len(str(m))
            log("=" * 65)
            log("  [!]  MERSENNE PRIME FOUND!  [!]")
            log(f"  M_{p:,}  ({decimal_digits:,} digits)  LL time: {fmt_duration(elapsed)}")
            log("=" * 65)
            prime_file = f"mersenne_prime_M{p}.txt"
            with open(prime_file, "w") as f:
                f.write(f"# Mersenne Prime M_{p} = 2^{p} - 1\n")
                f.write(f"# Decimal digits: {decimal_digits:,}\n")
```

```
                f.write(f"# Found: {datetime.now().isoformat()}\n\n")
                f.write(str(m))
            log(f"  Saved to: {prime_file}")
            found.append({"p": p, "digits": decimal_digits, "file": prime_file})
            if decimal_digits >= target_digits:
                log("  [TROPHY]  TARGET ACHIEVED!"); break
        else:
            log(f"    -> Composite (LL: {fmt_duration(elapsed)})")

        save_checkpoint({"last_p": p, "tested": tested, "found": found})
        if max_candidates and tested >= max_candidates: break

    log(f"\nTested {tested:,} candidates. Found: {len(found)}")


# ---------------------------------------------------------------------------
# Self-test
# ---------------------------------------------------------------------------
def run_selftest(backend: str = "python"):
    print("=" * 65)
    print(f"  SELF-TEST  (backend: {backend})")
    print("=" * 65)
    test_primes  = [p for p in KNOWN_MERSENNE_EXPONENTS if p <= 4423]
    non_mersenne = [4, 6, 8, 9, 10, 11, 14, 15, 16, 18, 20, 23, 25]
    passed = failed = 0
    print(f"\n{'Exponent':>12}  {'Expected':>8}  {'Got':>8}  {'Digits':>8}  {'Time':>8}")
    print("-" * 55)
    for p in test_primes + non_mersenne:
        expected = p in test_primes
        t0       = time.time()
        result   = lucas_lehmer(p, backend=backend)
        elapsed  = time.time() - t0
        ok       = result == expected
        passed  += ok; failed += not ok
        print(f"{p:>12}  {'prime' if expected else 'compos':>8}  "
              f"{'prime' if result else 'compos':>8}  "
              f"{digits_of(p):>8,}  {elapsed:>7.3f}s  {'OK' if ok else 'FAIL FAIL'}")
    print("-" * 55)
    print(f"\n{passed} passed, {failed} failed")
    if failed == 0: print("OK All tests passed!\n")
    else:           print("FAIL FAILURES.\n"); sys.exit(1)


# ---------------------------------------------------------------------------
# Benchmark
# ---------------------------------------------------------------------------
def run_benchmark():
    print("\n" + "=" * 65)
    print("  BENCHMARK: Backends + parallel sieve scaling")
    print("=" * 65)

    test_ps = [127, 521, 1279, 2203, 4253]
    backends_to_test = ["python"]
    if BackendInfo.gmpy2_available:  backends_to_test.append("gmpy2")
    if BackendInfo.cupy_available:   backends_to_test.append("cupy")
    if BackendInfo.numba_available:  backends_to_test.append("numba")

    for bk in backends_to_test:
        print(f"\n  -- Backend: {bk} {'-'*(42-len(bk))}")
        print(f"  {'Exponent':>10}  {'Digits':>8}  {'Time':>10}")
        print("  " + "-" * 35)
        timings = []
        for p in test_ps:
            t0 = time.time()
            lucas_lehmer(p, backend=bk)
            elapsed = time.time() - t0
            print(f"  {p:>10,}  {digits_of(p):>8,}  {elapsed:>9.4f}s")
            timings.append((p, elapsed))

        if len(timings) >= 2:
            p1, t1 = timings[-2]; p2, t2 = timings[-1]
            if t1 > 0 and t2 > 0:
```

```
                alpha = math.log(t2/t1) / math.log(p2/p1)
                C     = t1 / (p1 ** alpha)
                print(f"\n  Scaling alpha ~= {alpha:.2f}  (2.0=naive, 1.1=FFT)")
                for tgt in [TARGET_EXPONENT, 136_279_841]:
                    est = C * (tgt ** alpha)
                    print(f"  Est. p={tgt:,} ({digits_of(tgt):,}d): {fmt_duration(est)}")

    print(f"\n  -- Parallel sieve scaling {'-'*30}")
    print(f"  (100 candidates near p=10,000)")
    candidates = list(candidate_exponents(10_000))[:100]
    for n in [1, 2, mp.cpu_count()]:
        t0 = time.time()
        with mp.Pool(n) as pool:
            list(pool.imap_unordered(_sieve_worker, [(p, 1_000_000) for p in candidates]))
        print(f"  {n} worker(s): {time.time()-t0:.3f}s")
    print()



# ========================================================================
# DISTRIBUTED COMPUTING -- Coordinator / Worker
# ========================================================================
#
#  Architecture (one candidate tested per machine simultaneously):
#
#   +-------------------------------------------------------------+
#   |  Coordinator machine                                         |
#   |  python3 mersenne_prime_hunter.py --coordinator             |
#   |                                                              |
#   |  ? Generates candidate exponents in order                    |
#   |  ? Assigns each to a worker via XML-RPC                      |
#   |  ? Tracks in-progress work (reassigns if worker times out)  |
#   |  ? Collects results, saves checkpoint, logs prime finds      |
#   +----------------------+--------------------------------------+
#           network (LAN or internet)
#   +----------+----------+  +---------------------+
#   |  Worker machine 1   |  |  Worker machine 2   |  ...
#   |  python3 ...        |  |  python3 ...        |
#   |    --worker         |  |    --worker         |
#   |    --host 192.x.x.x |  |    --host 192.x.x.x |
#   |                     |  |                     |
#   |  Runs sieve + LL    |  |  Runs sieve + LL    |
#   |  on assigned p      |  |  on assigned p      |
#   +---------------------+  +---------------------+
#
#  Setup:
#   1. On coordinator:   python3 mersenne_prime_hunter.py --coordinator
#   2. On each worker:   python3 mersenne_prime_hunter.py --worker --host <coordinator-ip>
#
#  All flags (--backend, --workers, --digits) still work on worker machines.
#  The coordinator handles checkpointing and log rotation as normal.
#  Workers reconnect automatically if the coordinator restarts.
# ========================================================================


class CoordinatorServer:
    """
    XML-RPC server that manages distributed work assignment.
    Runs on the coordinator machine. Workers call these methods remotely.
    """

    def __init__(self, start_exp: int, target_digits: int):
        self._lock         = threading.Lock()
        self._next_p       = start_exp          # next exponent to assign
        self._in_progress  = {}                 # {p: (worker_id, assigned_at)}
        self._tested       = 0
        self._found        = []
        self._target_digits = target_digits
        self._done         = False

    # -- RPC methods (called by workers) -------------------------------------

```

```python
    def get_work(self, worker_id: str) -> dict:
        """
        Assign the next untested prime exponent to a worker.
        Returns {"p": int, "digits": int} or {"done": True} if finished.
        Also reassigns timed-out candidates from dead workers.
        """
        with self._lock:
            if self._done:
                return {"done": True}

            # Check for timed-out in-progress assignments first
            now = time.time()
            for p, (wid, assigned_at) in list(self._in_progress.items()):
                if now - assigned_at > WORK_TIMEOUT:
                    log(f"  [coordinator] Reassigning M_{p:,} "
                        f"(worker {wid} timed out after {WORK_TIMEOUT//60}min)")
                    self._in_progress[p] = (worker_id, now)
                    return {"p": p, "digits": digits_of(p)}

            # Find next prime exponent not already in progress
            p = self._next_p
            while True:
                if is_prime_miller_rabin(p) and p not in self._in_progress:
                    break
                p += 2
            self._next_p = p + 2

            self._in_progress[p] = (worker_id, time.time())
            self._tested += 1

            digs = digits_of(p)
            log(f"  [coordinator] Assigned M_{p:,} ({digs:,}d) -> worker {worker_id}")
            save_checkpoint({
                "last_p": p, "tested": self._tested, "found": self._found
            })
            return {"p": p, "digits": digs}

    def report_result(self, worker_id: str, p: int,
                      is_prime: bool, elapsed: float) -> bool:
        """Worker calls this when it finishes testing a candidate."""
        with self._lock:
            self._in_progress.pop(p, None)
            digs = digits_of(p)

            if is_prime:
                m              = (1 << p) - 1
                decimal_digits = len(str(m))
                log("=" * 65)
                log(f"  *** MERSENNE PRIME FOUND by worker {worker_id}! ***")
                log(f"  M_{p:,} = 2^{p:,} - 1")
                log(f"  Decimal digits: {decimal_digits:,}")
                log(f"  LL test time:   {fmt_duration(elapsed)}")
                log("=" * 65)
                prime_file = f"mersenne_prime_M{p}.txt"
                with open(prime_file, "w") as f:
                    f.write(f"# Mersenne Prime M_{p} = 2^{p} - 1\n")
                    f.write(f"# Decimal digits: {decimal_digits:,}\n")
                    f.write(f"# Found by worker: {worker_id}\n")
                    f.write(f"# Found: {datetime.now().isoformat()}\n\n")
                    f.write(str(m))
                log(f"  Saved to: {prime_file}")
                self._found.append({"p": p, "digits": decimal_digits,
                                    "file": prime_file})
                if decimal_digits >= self._target_digits:
                    log("  TARGET ACHIEVED!")
                    self._done = True
            else:
                log(f"  [coordinator] M_{p:,} ({digs:,}d) composite "
                    f"(worker {worker_id}, {fmt_duration(elapsed)})")

            save_checkpoint({
                "last_p": p, "tested": self._tested, "found": self._found
```

```python
            })
        return True

    def heartbeat(self, worker_id: str, p: int) -> bool:
        """Workers call this periodically during long LL tests to avoid timeout."""
        with self._lock:
            if p in self._in_progress:
                wid, _ = self._in_progress[p]
                self._in_progress[p] = (wid, time.time())
        return True

    def status(self) -> dict:
        """Return coordinator status (for monitoring)."""
        with self._lock:
            return {
                "next_p":       self._next_p,
                "in_progress":  len(self._in_progress),
                "tested":       self._tested,
                "found":        len(self._found),
                "done":         self._done,
            }


def run_coordinator(start_exp: int, target_digits: int, port: int):
    """Start the XML-RPC coordinator server."""
    server_obj = CoordinatorServer(start_exp, target_digits)

    server = xmlrpc.server.SimpleXMLRPCServer(
        ("0.0.0.0", port),
        logRequests=False,
        allow_none=True,
    )
    server.register_instance(server_obj)
    server.register_introspection_functions()

    # Resolve local IP robustly (gethostbyname fails on some Macs)
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        local_ip = s.getsockname()[0]
        s.close()
    except Exception:
        local_ip = "127.0.0.1"
    log("=" * 65)
    log("  MERSENNE PRIME HUNTER -- Coordinator Mode")
    log("=" * 65)
    log(f"  Listening on:    0.0.0.0:{port}")
    log(f"  Local IP:        {local_ip}")
    log(f"  Target:          {target_digits:,} decimal digits")
    log(f"  Starting from:   p = {start_exp:,}")
    log("")
    log("  On each worker machine run:")
    log(f"    python3 mersenne_prime_hunter.py --worker --host {local_ip}")
    log("")
    log("  Ctrl+C to stop.")
    log("")

    # Status thread -- prints a summary every 5 minutes
    def status_loop():
        while True:
            time.sleep(5 * 60)
            s = server_obj.status()
            log(f"  [status] next_p={s['next_p']:,}  in_progress={s['in_progress']}  "
                f"tested={s['tested']:,}  found={s['found']}")

    t = threading.Thread(target=status_loop, daemon=True)
    t.start()

    try:
        server.serve_forever()
    except KeyboardInterrupt:
        log("  Coordinator shutting down.")
```

```python


def run_worker(coordinator_host: str, coordinator_port: int,
               backend: str, n_workers: int):
    """
    Connect to coordinator, pull candidates, run sieve+LL, report results.
    Automatically reconnects if the coordinator goes down temporarily.
    """
    url       = f"http://{coordinator_host}:{coordinator_port}"
    worker_id = f"{socket.gethostname()}-{os.getpid()}"

    BackendInfo.detect()
    log("=" * 65)
    log("  MERSENNE PRIME HUNTER -- Worker Mode")
    log("=" * 65)
    log(f"  Worker ID:    {worker_id}")
    log(f"  Coordinator:  {url}")
    log(f"  Backend:      {backend}")
    log(f"  Local cores:  {n_workers}")
    log("")

    def connect():
        while True:
            try:
                proxy = xmlrpc.client.ServerProxy(url, allow_none=True)
                proxy.status()   # test connection
                log(f"  Connected to coordinator at {url}")
                return proxy
            except Exception as e:
                log(f"  Cannot reach coordinator ({e}) -- retrying in 10s...")
                time.sleep(10)

    proxy = connect()

    while True:
        # -- Get next assignment ----------------------------------------------
        try:
            work = proxy.get_work(worker_id)
        except Exception as e:
            log(f"  Lost connection to coordinator ({e}) -- reconnecting...")
            proxy = connect()
            continue

        if work.get("done"):
            log("  Coordinator says search is complete. Exiting.")
            break

        p    = work["p"]
        digs = work["digits"]
        log(f"  Assigned: M_{p:,} ({digs:,} digits)")

        # -- Step 1: trial factor sieve (local, fast) -------------------------
        t0 = time.time()
        if has_small_factor(p, trial_limit=10_000_000):
            elapsed = time.time() - t0
            log(f"    Composite (trial factor, {elapsed:.2f}s) -- reporting...")
            try:
                proxy.report_result(worker_id, p, False, elapsed)
            except Exception:
                pass
            continue

        # -- Step 2: Lucas-Lehmer test ----------------------------------------
        log(f"    Passed sieve -- running Lucas-Lehmer ({backend})...")

        # Heartbeat thread: prevents coordinator timeout during long LL tests
        stop_hb   = threading.Event()
        def heartbeat_loop():
            while not stop_hb.is_set():
                try:
                    proxy.heartbeat(worker_id, p)
                except Exception:
```

```
                    pass
                time.sleep(WORK_TIMEOUT // 4)

        hb_thread = threading.Thread(target=heartbeat_loop, daemon=True)
        hb_thread.start()

        t1       = time.time()
        tracker  = LLProgressTracker(p, 0)
        is_prime = lucas_lehmer(p, backend=backend, progress_cb=tracker.update)
        elapsed  = time.time() - t1

        stop_hb.set()

        # -- Report result to coordinator ------------------------------------
        while True:
            try:
                proxy.report_result(worker_id, p, is_prime, elapsed)
                break
            except Exception as e:
                log(f"  Failed to report result ({e}) -- retrying in 10s...")
                time.sleep(10)
                try:
                    proxy = connect()
                except Exception:
                    pass


# ---------------------------------------------------------------------------
# Entry point
# ---------------------------------------------------------------------------
def main():
    BackendInfo.detect()

    parser = argparse.ArgumentParser(
        description="Search for large Mersenne primes (GPU + multiprocessing).",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=__doc__,
    )
    parser.add_argument("--selftest",  action="store_true")
    parser.add_argument("--benchmark", action="store_true")
    parser.add_argument("--start",     type=int,   default=None)
    parser.add_argument("--digits",    type=int,   default=TARGET_DIGITS)
    parser.add_argument("--max",       type=int,   default=None, dest="max_candidates")
    parser.add_argument("--demo",      action="store_true")
    parser.add_argument("--no-resume", action="store_true")
    parser.add_argument("--backend",   default="auto",
                        choices=["auto","cpu","gpu","python","gmpy2","cupy","numba"])
    parser.add_argument("--workers",   type=int,   default=None,
                        help="Total worker processes (default: all CPU cores).")
    parser.add_argument("--coordinator", action="store_true",
                        help="Run as coordinator (assigns work to remote workers).")
    parser.add_argument("--worker",    action="store_true",
                        help="Run as worker (connects to coordinator for candidates).")
    parser.add_argument("--host",      type=str,   default="127.0.0.1",
                        help="Coordinator IP/hostname (used by --worker).")
    parser.add_argument("--port",      type=int,   default=COORDINATOR_PORT,
                        help=f"Coordinator port (default: {COORDINATOR_PORT}).")
    args = parser.parse_args()

    print(BackendInfo.report())

    # Resolve backend
    force   = args.backend if args.backend != "auto" else None
    backend = BackendInfo.best_backend(force)
    print(f"\n  Active backend: {backend}\n")

    total_workers = args.workers or BackendInfo.cpu_count
    n_sieve       = max(1, total_workers - 1)
    n_ll          = max(1, total_workers // 2)

    if args.selftest:
        run_selftest(backend); return
```

```
    if args.benchmark:
        run_benchmark(); return

    # Distributed modes
    if args.coordinator:
        target_digits = args.digits
        if args.start:
            start_exp = args.start
        elif not args.no_resume and os.path.exists(CHECKPOINT_FILE):
            cp        = load_checkpoint()
            start_exp = cp["last_p"] + 2
            log(f"Coordinator resuming from checkpoint: p={start_exp:,}")
        else:
            start_exp = math.ceil(target_digits / math.log10(2))
        run_coordinator(start_exp, target_digits, args.port)
        return

    if args.worker:
        n_workers = args.workers or BackendInfo.cpu_count
        run_worker(args.host, args.port, backend, n_workers)
        return


    target_digits = args.digits
    if args.demo:
        start_exp = 10_000
        log("DEMO MODE: searching near p=10,000")
    elif args.start:
        start_exp = args.start
    elif not args.no_resume and os.path.exists(CHECKPOINT_FILE):
        cp        = load_checkpoint()
        start_exp = cp["last_p"] + 2
        log(f"Resuming from checkpoint: last_p={cp['last_p']:,}")
    else:
        start_exp = math.ceil(target_digits / math.log10(2))

    # ------------------------------------------------------------------
    # 2-minute stop/restart cycle + 24-hour log rotation
    # ------------------------------------------------------------------
    # Each cycle: launch the search as a child process, let it run for
    # RESTART_INTERVAL seconds, then SIGTERM it (triggering checkpoint),
    # reload the checkpoint, and re-launch. Benefits:
    #   - Keeps memory clean (no GC/GIL buildup over long runs)
    #   - A crash loses at most 2 minutes of work
    #   - Child processes exit cleanly between cycles
    # ------------------------------------------------------------------
    cycle         = 0
    last_rotation = time.time()   # track when we last rotated the log
    while True:
        cycle += 1
        log(f"  \u2500\u2500 Cycle {cycle} - running for {RESTART_INTERVAL//60} min then
 checkpointing \u2500\u2500")

        # Build child command (--_inner skips the restart loop in the child)
        cmd = [
            sys.executable, os.path.abspath(__file__),
            "--backend", backend,
            "--workers", str(total_workers),
            "--digits",  str(target_digits),
            "--_inner",
        ]
        if args.max_candidates:
            cmd += ["--max", str(args.max_candidates)]

        proc       = subprocess.Popen(cmd)
        cycle_t0   = time.time()

        # Poll until deadline or child finishes on its own
        while True:
            if time.time() - cycle_t0 >= RESTART_INTERVAL:
                log(f"  {RESTART_INTERVAL//60}-min cycle complete -- checkpointing and
 restarting...")
```

```
                proc.terminate()
                try:
                    proc.wait(timeout=15)
                except subprocess.TimeoutExpired:
                    proc.kill(); proc.wait()
                break
            ret = proc.poll()
            if ret is not None:            # child exited on its own
                if ret == 0:
                    log("  Search finished (prime found or --max reached).")
                else:
                    log(f"  Child exited with code {ret}.")
                return
            time.sleep(2)

        # Reload checkpoint so the next cycle resumes where we left off
        if os.path.exists(CHECKPOINT_FILE):
            cp        = load_checkpoint()
            start_exp = cp["last_p"] + 2
            tested_count = cp["tested"]
            log(f"  Resuming from p={start_exp:,}  ({tested_count:,} candidates tested so far)")
        else:
            log("  No checkpoint found -- next cycle restarts from beginning.")

        log("  Restarting in 3 seconds...")
        time.sleep(3)

        # Check if 24-hour log rotation is due
        if time.time() - last_rotation >= LOG_ROTATE_INTERVAL:
            log("  24-hour mark reached -- rotating log file...")
            rotate_log()
            last_rotation = time.time()


# ---------------------------------------------------------------------------
# Inner search (runs inside child processes spawned by the restart loop)
# ---------------------------------------------------------------------------
def main_inner():
    """One cycle of search, called by child processes. No restart wrapper."""
    BackendInfo.detect()

    parser = argparse.ArgumentParser(add_help=False)
    parser.add_argument("--backend", default="auto",
                        choices=["auto","cpu","gpu","python","gmpy2","cupy","numba"])
    parser.add_argument("--workers", type=int, default=None)
    parser.add_argument("--digits",  type=int, default=TARGET_DIGITS)
    parser.add_argument("--max",     type=int, default=None, dest="max_candidates")
    args, _ = parser.parse_known_args()

    force         = args.backend if args.backend != "auto" else None
    backend       = BackendInfo.best_backend(force)
    total_workers = args.workers or BackendInfo.cpu_count
    n_sieve       = max(1, total_workers - 1)
    n_ll          = max(1, total_workers // 2)
    target_digits = args.digits

    # Always resume from checkpoint in inner runs
    if os.path.exists(CHECKPOINT_FILE):
        cp        = load_checkpoint()
        start_exp = cp["last_p"] + 2
        log(f"  [inner] Resuming from p={start_exp:,}")
    else:
        start_exp = math.ceil(target_digits / math.log10(2))

    if total_workers == 1:
        run_search_single(start_exp, target_digits, backend, args.max_candidates)
    else:
        run_search_parallel(start_exp, target_digits, backend,
                            n_sieve, n_ll, args.max_candidates)


if __name__ == "__main__":
```

```
    if "--_inner" in sys.argv:
        sys.argv.remove("--_inner")
        main_inner()
    else:
        main()
```

Listing 1. mersenne_prime_hunter.py — complete source

Independent Researcher
Email address: jenkj@unc.edu